DEVELOPER PLAYBOOK

Narri — Day-by-Day Build Plan + Solo Developer Speed Toolkit

Backend & Frontend tasks for every day, plus everything that will make you faster

MERN Stack · Solo Developer · 30-Day Sprint
Written like a mentor sitting next to you — August 2026

Table of Contents

Before Day 1: Read This First

I'm going to talk to you the way I'd talk to a junior developer I'm mentoring — not just handing you a task list, but explaining *why* each thing matters and *how* to do it without wasting hours. Read this chapter once, fully, before you touch Day 2. Everything in here is designed to save you real time across the next 29 days.

Why most solo 30-day builds fail

Not because the developer isn't skilled — because they rebuild things that already exist. Every hour spent writing a custom dropdown component, a custom form validator, or a custom loading spinner is an hour not spent on what makes *your* product different. Your job for the next 29 days is to be ruthless about using existing, battle-tested tools for anything generic, and to spend your own thinking time only on Narri-specific logic: your product catalog, your checkout, your admin controls.

1. Editor setup — do this once, benefit every day

Tool	Why it saves you time
VS Code + **ES7+ React Snippets**	Type `rafce` and hit tab → full React component boilerplate instantly. No more typing the same function/export skeleton 40 times.
Tailwind CSS IntelliSense	Autocompletes class names and shows the actual color/spacing as you type — stops you tabbing to the docs constantly.
Thunder Client (VS Code extension)	Test your API endpoints without leaving the editor or opening Postman. Save requests per route, reuse them daily.
MongoDB for VS Code	Browse and edit your Atlas collections visually, right inside the editor — no need to open the Atlas website constantly.
Prettier + ESLint	Auto-formats on save. You never manually align brackets or debug a missing semicolon again.
GitLens	See who/when changed a line inline — mostly useful for you to track your own history, and essential if you ever add a collaborator.
Auto Rename Tag + Path Intellisense	Renaming a JSX tag updates the closing tag automatically; import paths autocomplete instead of you typing them from memory.

2. Libraries that replace days of your own code

Instead of building...	Use...	Why
Form validation by hand	**react-hook-form** + **zod**	Declarative validation rules, built-in error states, minimal re-renders. Saves ~a full day across all your forms.
Manual fetch + loading/error state for every API call	**TanStack Query (React Query)**	Handles caching, loading, error, and refetching automatically. You write 5 lines instead of 25 per data fetch.
Redux boilerplate for cart/auth state	**Zustand**	A cart store in ~15 lines, no actions/reducers/providers ceremony.
Custom icon SVGs	**lucide-react**	Hundreds of consistent, free icons as React components — `import { ShoppingCart } from 'lucide-react'`.
Building buttons/modals/dropdowns from scratch	**shadcn/ui** components on top of Tailwind	Accessible, pre-styled, and you own the code (copy-paste, not a black-box dependency) — customize freely.
Image upload handling + resizing	**Cloudinary** upload widget/SDK	Drag-drop upload, automatic compression and CDN delivery — don't write your own file storage logic.
Writing your own JWT/bcrypt logic from scratch	**jsonwebtoken** + **bcryptjs** npm packages	These are the industry-standard, audited implementations — never hand-roll crypto/auth logic.

3. Stop manually testing everything — automate the boring parts

> **Seed script:** Write one `seed.js` script in week 1 that inserts 20-30 dummy products, a few categories, and a test admin user into MongoDB with one command (`node seed.js`). Every time you need test data, run it — don't manually re-add products through the admin UI ten times a day.

> **Save your API calls in Thunder Client/Postman as a collection.** The first time you write a login request, save it. Every day after, you re-run it in one click instead of retyping headers and JSON bodies.

> **Use faker.js** to generate realistic fake names, addresses, and prices for your seed data instead of typing "Test Product 1", "Test Product 2" by hand.

4. Git & deployment habits that remove entire categories of stress

Connect your GitHub repo to Vercel (frontend/admin) and Render (backend) on **Day 1**, not Day 29. Once connected, every `git push` auto-deploys. This means you're never doing a stressful manual deployment at the end — you've already deployed successfully hundreds of times by accident, so Day 29 is a non-event.

Commit small and often — after every working feature, not once a day. If something breaks, `git log` shows you exactly which 20-minute chunk of work introduced the bug, instead of hunting through a full day's changes.

5. Time-boxing — the biggest speed hack of all

Set a timer for each task in the daily plan below. If a task is taking 2x longer than planned, stop, ship the simplest version that works, and move on — polish is a Week 4 activity, not a Week 1 activity. Perfectionism on Day 6's product card design is the #1 reason solo 30-day builds slip.

6. Using AI coding tools well

Use an AI assistant (Claude Code, Copilot, etc.) to generate boilerplate — a CRUD controller skeleton, a form component shell, a Tailwind layout. But always read and understand what it gives you before committing it; you are the one debugging it at 11pm on Day 24, not the AI. Treat it like a very fast junior developer whose work you review, not an autopilot.

Week 1 — Foundation, Auth, Product Core

This week you're building the skeleton everything else hangs on. Nothing here is visually impressive yet — resist the urge to make it pretty. A boring, working auth system beats a beautiful broken one.

Day 1

Backend: Init Express server, connect MongoDB Atlas, set up folder structure (models/controllers/routes/middleware). Deploy an empty "hello world" API to Render.

Frontend: Init React with Vite, install and configure Tailwind, set up folder structure. Deploy an empty app to Vercel.

Teacher's note: deploying an empty app on Day 1 feels pointless — it isn't. You're proving the entire pipeline (GitHub → Render/Vercel → live URL) works before you have anything real riding on it. Debugging a broken deploy pipeline on Day 29 with a client waiting is a nightmare; debugging it on Day 1 with nothing at stake takes 20 minutes.

Day 2

Backend: Define Mongoose schemas: User, Product, Category, Order, Review, Banner (see the schema document from earlier for exact fields).

Frontend: Set up React Router, build empty page shells: Home, Product, Cart, Checkout, Login/Signup.

Teacher's note: writing all your schemas in one sitting — even before you need most of them — means every later feature just plugs into a structure you already understand, instead of you redesigning the data model mid-sprint.

Day 3

Backend: Auth APIs — signup, login, bcrypt password hashing, JWT generation.

Frontend: Signup & Login page UI — forms with react-hook-form, client-side validation.

Teacher's note: use bcryptjs and jsonwebtoken — do not write your own hashing or token logic. This is the one place in the whole app where "reinventing the wheel" is actually a security risk, not just wasted time.

Day 4

Backend: Auth middleware — protect (verifies JWT) and adminOnly (checks role).

Frontend: Connect signup/login forms to the API, store the token, build an AuthContext and a protected-route wrapper component.

Teacher's note: build the protected-route wrapper once, generically (it should accept "which roles are allowed" as a prop). You'll reuse this exact component for every customer-only and admin-only page for the rest of the build — don't rewrite the logic each time.

Day 5

Backend: Product CRUD APIs (create/list/detail/update/delete) + Category CRUD APIs.

Frontend: Homepage — fetch and render the product grid using TanStack Query.

Teacher's note: this is your first real use of React Query — notice how little code it takes to get loading and error states for free. Every future data-fetching screen (orders, reviews, banners) follows this exact same pattern, so once this "clicks" today, every later day gets faster.

Day 6

Backend: Add filtering/sorting/pagination query params to the Product API.

Frontend: Product detail page — image gallery, price, description, size selector (static for now).

Day 7 — BUFFER

Backend: Fix any auth bugs, run every Week 1 endpoint through your Thunder Client collection.

Frontend: Fix UI bugs, do a quick mobile-width check on everything built so far (resize your browser, don't skip this).

Teacher's note: use this day to write your seed.js script if you haven't yet — it pays for itself starting next week when you need realistic test data constantly.

Week 2 — Storefront: Browse, Cart, Checkout, Payment

This week the site becomes actually usable end-to-end. Payment integration is the trickiest part of the whole build — give yourself permission to spend extra buffer time here if needed.

Day 8

Backend: Add a text search index and search endpoint on Product.

Frontend: Wire category filters, price filter, and sort dropdown to the Product API.

Day 9

Backend: Review the Order schema fields — no new endpoints needed yet (cart lives client-side).

Frontend: Build cart state with Zustand + localStorage persistence; build the Cart page UI.

Teacher's note: keeping cart state entirely client-side (no "cart" API/collection) is a deliberate simplification — it removes an entire category of backend work and sync bugs. Don't build a server-side cart unless a real requirement forces it later.

Day 10

Backend: Order creation API — POST /orders, calculates itemsPrice/shippingPrice/totalPrice server-side.

Frontend: Checkout page — address form + order summary UI.

Teacher's note: always recalculate the price on the server from the actual product prices in the database — never trust a price sent from the frontend. This is a common security hole in beginner e-commerce builds.

Day 11

Backend: Razorpay/Stripe integration — create a payment order/intent endpoint.

Frontend: Connect checkout submit to the create-order API, integrate the payment widget SDK.

Day 12

Backend: Payment webhook handler — verify signature, mark order paid, update status.

Frontend: Order confirmation page; handle payment success/failure redirect states.

Teacher's note: always trust the webhook, not the frontend redirect, to mark an order as paid. A user closing their browser right after paying shouldn't leave their order stuck as "pending" forever — the webhook is what fires reliably from the payment provider's side.

Day 13

Backend: Order APIs — GET /orders/mine, GET /orders/:id.

Frontend: "My Orders" page — list view + single order detail view.

Day 14 — BUFFER

Backend: Test the full payment flow with test API keys, fix webhook edge cases.

Frontend: Test the full purchase journey (browse → cart → pay → confirmation), fix bugs.

Teacher's note: this is the single most important buffer day in the whole plan. If payment isn't rock-solid by tonight, don't move on to admin features tomorrow — extend this buffer by pulling from Week 3's slack instead.

Week 3 — Admin Dashboard: Products, Banners, Reviews

Good news: almost everything here reuses patterns you already built in Week 1-2 (protected routes, React Query fetching, forms). This week should feel noticeably faster than the last two.

Day 15

Backend: Harden admin middleware; manually test with a non-admin token to confirm it actually blocks.

Frontend: Admin dashboard shell — sidebar nav, layout, protected admin-only routes.

Day 16

Backend: Add a stock-update endpoint if missing; confirm the Product API covers everything admin needs.

Frontend: Admin Product management UI — list, add, edit, delete, stock field.

Teacher's note: reuse your Product API from Week 1 — you don't need separate "admin product" endpoints, just protect the write operations (create/update/delete) with adminOnly middleware.

Day 17

Backend: Banner CRUD API + Cloudinary image upload endpoint.

Frontend: Admin Banner management UI — upload image, set position/order, toggle active.

Day 18

Backend: Add GET /banners?active=true sorted by order.

Frontend: Connect the homepage hero section to the live banner API (carousel).

Day 19

Backend: Review API — create review, list by product, approve/reject flag.

Frontend: Reviews section on the product page — display approved reviews + submit-review form.

Day 20

Backend: Refine review approval logic (one review per user per product — enforced by a unique compound index).

Frontend: Admin Review moderation UI — table with approve/reject/delete actions.

Day 21 — BUFFER

Backend: Fix banner/review API bugs.

Frontend: Test banner + review flows end-to-end, polish admin UI.

Week 4 — Orders/Sales Dashboard, Polish, Launch

This week is about reliability, not new features. If you're tempted to add "just one more thing," don't — that's exactly how launches slip.

Day 22

Backend: Admin Orders API — list all orders (filter by status), update order status.

Frontend: Admin Orders page — table, status filter, "update status" action.

Day 23

Backend: Sales summary API — aggregate total revenue, order count (MongoDB aggregation pipeline).

Frontend: Admin dashboard stat cards — revenue, order count, pending reviews, active banners.

Day 24

Backend: Add any missing DB indexes, quick performance pass on slow queries.

Frontend: Mobile responsiveness pass — storefront (home, product, cart, checkout).

Day 25

Backend: Standardize API error responses — one consistent JSON shape, no leaking stack traces.

Frontend: Mobile responsiveness pass — admin dashboard; add loading/error states everywhere.

Day 26

Backend + Frontend together: Full manual QA — signup → browse → cart → checkout → payment → order visible in admin. Log every bug you hit as you go, fix as many as you can same-day.

Day 27

Backend + Frontend: Fix remaining bugs from Day 26. Cross-browser/device check (Chrome + Safari mobile especially, since your traffic is Instagram-driven and mostly iOS Safari).

Day 28 — BUFFER

Backend + Frontend: Final catch-up day — absorb anything still open before launch prep begins.

Day 29

Backend: Set production env vars, deploy final backend build, test one real (small) transaction end-to-end.

Frontend: Production build, deploy frontend + admin, verify the live site end-to-end.

Day 30

Backend + Frontend: Client walkthrough, handover, keep the rest of the day free as a buffer for last-minute fixes.

Quick Reference Cheatsheet

Commands you'll type constantly

```
git add . && git commit -m "message" && git push
npm run dev                          # start dev server
node seed.js                         # repopulate test data
npm install <package> --save         # add a dependency
```

Handy VS Code snippet triggers (ES7+ React Snippets)

Trigger	Produces
`rafce`	React arrow function component with default export
`useState`	useState hook snippet
`useEffect`	useEffect hook snippet

End-of-day habit (do this every single day)

1. Commit and push your working code. 2. Quickly re-run your Thunder Client collection to confirm nothing broke. 3. Glance at tomorrow's row in this plan so you start the next day already knowing the goal.

> You already have the harder documents (SRS, ER diagram, schema, API reference) from earlier — keep this plan open alongside them as your daily driver, and refer back to those for exact field names and endpoint shapes as you build each day's task.